

Agent Harness Engineering

APRIL 19, 2026

A coding agent is the model plus everything you build around it. Harness engineering treats that scaffolding as a real artifact, and it tightens every time the agent slips.

Roughly: anytime you find an agent makes a mistake, you take the time to engineer a solution such that the agent never makes that mistake again.

We've spent the last two years arguing about models. Which one is smartest, which one writes the cleanest React, which one hallucinates less. That conversation is fine as far as it goes, but it's missing the other half of the system. The model is one input into a running agent. The rest is the *harness*: the prompts, tools, context policies, hooks, sandboxes, subagents, feedback loops, and recovery paths wrapped around the model so it can actually finish something.

A decent model with a great harness beats a great model with a bad harness. I've watched this play out on my own work over and over. And increasingly the interesting engineering isn't in picking the model, it's in designing the scaffolding around it.

That discipline now has a name. Viv Trivedy coined the term *harness engineering*, and his "[Anatomy of an Agent Harness](#)" post is the cleanest derivation of what a harness actually is and why each piece exists. [Dex Horthy](#) has been tracking the pattern as it emerges. [HumanLayer](#) frames most agent failures as "skill issues" that come down to configuration rather than model weights. [Anthropic's engineering team](#) has published what I think is the best public breakdown of how to design a harness for long-running work. And [Birgitta Böckeler](#) has a good overview of what this looks like from the user's

side.

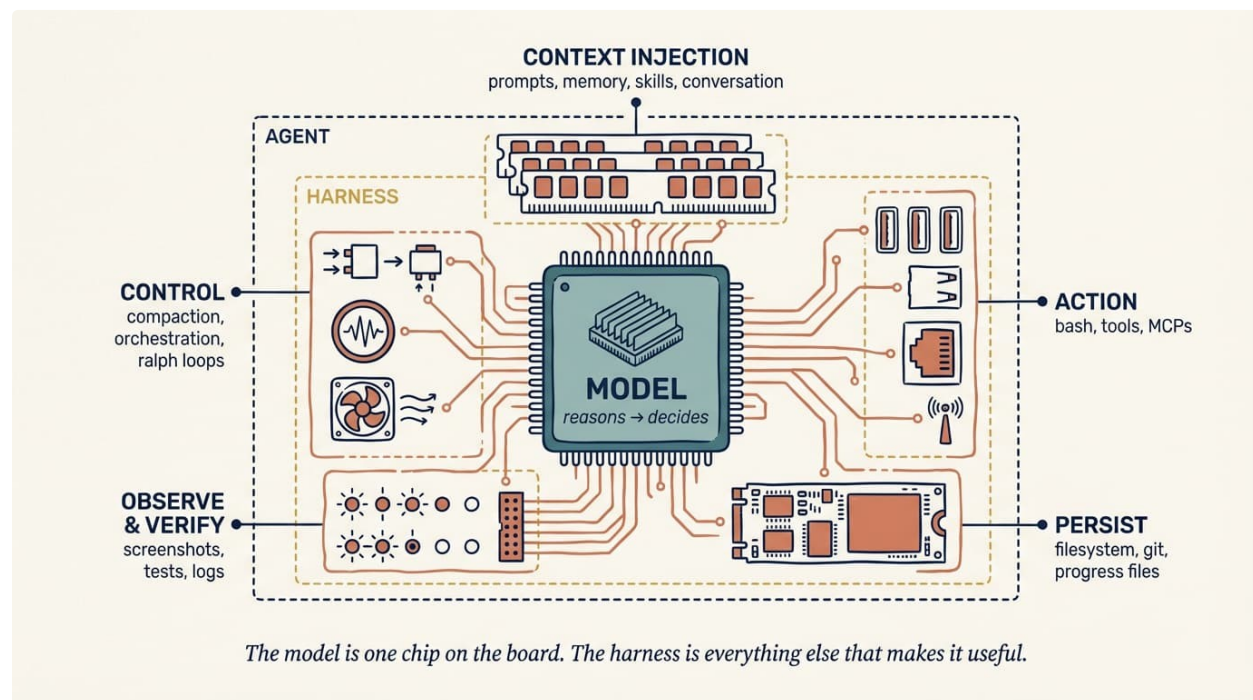
This post is my attempt to pull those threads together.

What is a harness, really?

Viv's one-liner does most of the work:

Agent = Model + Harness. If you're not the model, you're the harness.

A harness is every piece of code, configuration, and execution logic that isn't the model itself. A raw model is not an agent. It becomes one once a harness gives it state, tool execution, feedback loops, and enforceable constraints.



Concretely, a harness includes:

- System prompts, CLAUDE.md, AGENTS.md, skill files, and subagent prompts
- Tools, skills, MCP servers, and their descriptions
- Bundled infrastructure (filesystem, sandbox, browser)

- Orchestration logic (subagent spawning, handoffs, model routing)
- Hooks and middleware for deterministic execution (compaction, continuation, lint checks)
- Observability (logs, traces, cost and latency metering)

[Simon Willison](#) reduces the loop part to its essence: an agent is a system that “*runs tools in a loop to achieve a goal.*” The skill is in the design of both the tools and the loop.

If that sounds like a lot of surface area, it is. And it’s *your* surface area, not the model provider’s. Claude Code, Cursor, Codex, Aider, Cline: these are all harnesses. The model underneath is sometimes the same, but the behaviour you experience is dominated by what the harness does.

```
coding agent = AI model(s) + harness
```

This equation, articulated by Viv and echoed by HumanLayer, is where the work actually lives. The debate over the left-hand side is loud. Most of the actual leverage sits on the right.

The “skill issue” reframe

There’s a pattern I watch engineers fall into. The agent does something dumb, the engineer blames the model, and the blame gets filed under “wait for the next version.”

The harness-engineering mindset rejects that default. The failure is usually legible. The agent didn’t know about a convention, so you add it to `AGENTS.md`. The agent ran a destructive command, so you add a hook that blocks it. The agent got lost in a 40-step task, so you split it into a planner and an executor. The agent kept “finishing” broken code, so you wire a typecheck back-pressure signal into the loop.

HumanLayer says: “*it’s not a model problem. It’s a configuration problem.*” Harness engineering is what happens when you take that seriously.

There’s a striking data point that shows up in both Viv’s write-up and HumanLayer’s. On

Terminal Bench 2.0, Claude Opus 4.6 running inside Claude Code scores far lower than the same model running in a custom harness. Viv's team moved a coding agent from Top 30 to Top 5 by changing only the harness. Models get post-training coupled to the harness they were trained against. Moving them into a different harness, with better tools for your codebase, a tighter prompt, and sharper back-pressure, can unlock capability the original harness was leaving on the floor.

This is the opposite of the “just wait for GPT-6” narrative. The gap between what today's models can do and what you see them doing is largely a harness gap.

The ratchet: every mistake becomes a rule

The most important habit in harness engineering is treating agent mistakes as permanent signals. Not one-off stories to laugh about, not “bad runs” to retry. Signals.

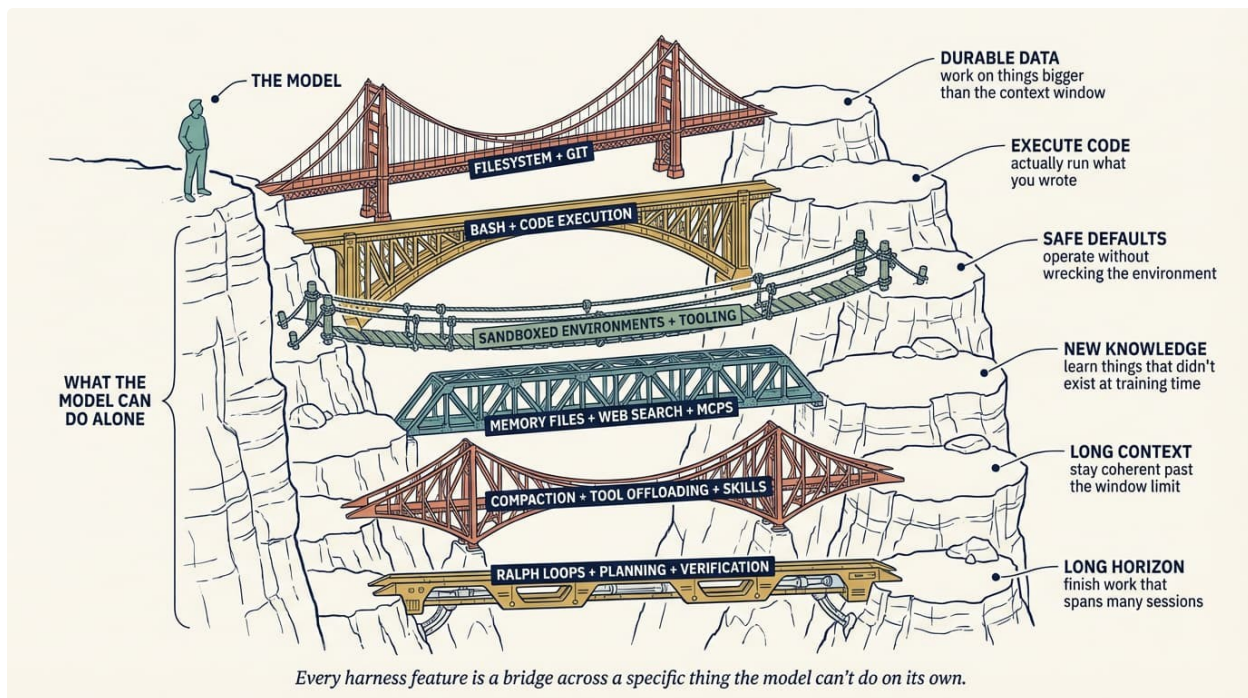
If the agent ships a PR with a commented-out test and I merge it by accident, that's an input. The next version of my `AGENTS.md` says “never comment out tests; delete them or fix them.” The next version of my pre-commit hook greps for `.skip(` and `xit(` in the diff. The next version of my reviewer subagent flags commented-out tests as a blocker.

You only add constraints when you've seen a real failure. You only remove them when a capable model has made them redundant. Every line in a good `AGENTS.md` should be traceable back to a specific thing that went wrong.

This is also why harness engineering is a discipline rather than a framework. The right harness for your codebase is shaped by your failure history. You can't download it.

Working backwards from behaviour

The framing from Viv that I find most useful when I'm actually designing a harness is to start from the behaviour you want and derive the harness piece that delivers it. His pattern: *behaviour we want (or want to fix) → harness design to help the model achieve this.*



The useful thing about deriving it this way is that every harness component has a specific job. If you can't name the behaviour a component exists to deliver, it probably shouldn't be there.

The rest of this section walks the pieces in roughly the order Viv does, with the specific patterns I've found worth stealing.

Filesystem and Git: durable state

The filesystem is the most foundational primitive, and it tends to be underrated because it's boring. Models can only directly operate on what fits in context. Without a filesystem, you're copy-pasting into a chat window, and that isn't a workflow.

Once you have a filesystem, the agent gets a workspace to read data, code, and docs; a place to offload intermediate work instead of holding it in context; and a surface where multiple agents and humans can coordinate through shared files. Adding Git on top gives you versioning for free, so the agent can track progress, roll back errors, and branch experiments.

Most of the other harness primitives end up pointing at the filesystem for something.

Bash and code execution: the general-purpose tool

The main agent loop today is a ReAct loop: the model reasons, takes an action via a tool call, observes the result, and repeats. But a harness can only execute the tools it has logic for. You can try to pre-build a tool for every possible action, or you can give the agent bash and let it build the tools it needs on the fly.

Willison's take on this is that agents already excel at shell commands; most tasks collapse to a few well-chosen CLI invocations. Harnesses still ship focused tools, but bash plus code execution has become the default general-purpose strategy for autonomous problem solving. It's the difference between teaching someone to use a single kitchen gadget and handing them a kitchen.

Sandboxes and default tooling

Bash is only useful if it runs somewhere safe. Running agent-generated code on your laptop is risky, and a single local environment doesn't scale to many parallel agents.

Sandboxes give agents an isolated operating environment. Instead of executing locally, the harness connects to a sandbox to run code, inspect files, install dependencies, and verify work. You can allow-list commands, enforce network isolation, spin up new environments on demand, and tear them down when the task is done.

A good sandbox ships with good defaults: pre-installed language runtimes and packages, Git and test CLIs, a headless browser for web interaction. Browsers, logs, screenshots, and test runners are what let the agent observe its own work and close the self-verification loop.

The model doesn't configure its execution environment. Deciding where the agent runs, what's available, and how it verifies its output are all harness-level calls.

Memory and search: continual learning

Models have no additional knowledge beyond their weights and what's currently in context. Without the ability to edit weights, the only way to add knowledge is through context injection.

The filesystem is again the primitive. Harnesses support memory file standards like `AGENTS.md` that get injected on every start. As the agent edits that file, the harness

reloads it, and knowledge from one session carries into the next. This is a crude but effective form of continual learning.

For knowledge that didn't exist at training time (new library versions, current docs, today's data) web search and MCP tools like Context7 bridge the cutoff. These are useful primitives to bake into the harness rather than leaving to the user.

Battling context rot

Context rot is the observation that models get worse at reasoning and completing tasks as the context window fills up. Context is scarce, and harnesses are largely delivery mechanisms for good context engineering.

Three techniques show up repeatedly:

Compaction. When the window gets close to full, something has to give. Letting the API error is not an option for a production harness, so the harness intelligently summarizes and offloads older context so the agent can keep working.

Tool-call offloading. Large tool outputs (think 2,000-line log files) clutter context without adding much signal. The harness keeps the head and tail tokens above a threshold and offloads the full output to the filesystem, where the agent can read it on demand.

Skills with progressive disclosure. Loading every tool and MCP into context at startup degrades performance before the agent takes a single action. Skills let the harness reveal instructions and tools only when the task actually calls for them.

Anthropic's harness post adds one more technique for the really long jobs: full context resets, where the harness tears the session down and rebuilds it from a compact hand-off file. They're explicit that *compaction alone wasn't sufficient* for long tasks; sometimes you need to start fresh with a structured brief. This is closer to how humans onboard a new engineer than to how we usually think about "memory."

Long-horizon execution: Ralph Loops, planning, verification

Autonomous long-horizon work is the holy grail and the hardest thing to get right. Today's models suffer from early stopping, poor decomposition of complex problems, and incoherence as work stretches across multiple context windows. The harness has

to design around all of that.

I've written about autonomous coding loops like the Ralph Loop before in [self-improving agents](#) and in my [2026 trends piece](#), but it's worth restating in this framing: a hook intercepts the model's attempt to exit and re-injects the original prompt into a fresh context window, forcing the agent to continue against a completion goal. Each iteration starts clean but reads state from the previous one through the filesystem. It's a surprisingly simple trick for turning a single-session agent into a multi-session one, and it's the kind of primitive you'd never derive from "just use a smarter model."

Planning is when the model decomposes a goal into a sequence of steps, usually into a plan file on disk. The harness supports this with prompting and reminders about how to use the plan file. After each step, the agent checks its work via self-verification: hooks run a pre-defined test suite and loop failures back to the model with the error text, or the model reviews its own output against explicit criteria.

Planner / generator / evaluator splits. Anthropic's long-running harness work is explicit that separating generation from evaluation into distinct agents outperforms self-evaluation, because agents reliably skew positive when grading their own work. It's GANs for prose. The related pattern is the sprint contract, where the generator and evaluator negotiate what "done" actually means before code gets written. In my own workflows, writing down the done-condition before starting has caught more scope drift than any prompt change I've ever made.

Hooks: the enforcement layer

Hooks are what separate "I told the agent to do X" from "the system enforces X."

A hook is a script that runs at a specific lifecycle point: before a tool call, after a file edit, before commit, on session start. They're the right place for things the agent should never forget but often does. Run typecheck and lint and tests after every edit and surface failures. Block destructive bash (`rm -rf`, `git push --force`, `DROP TABLE`). Require approval before opening a PR or pushing to `main`. Auto-format on write so the agent doesn't waste tokens on whitespace.

The principle HumanLayer highlights and I've come to agree with is: success is silent, failures are verbose. If typecheck passes, the agent hears nothing. If it fails, the error

text gets injected into the loop and the agent self-corrects. That makes the feedback loop almost free in the common case and directly actionable when something goes wrong.

AGENTS.md and tool choice

The flat markdown rulebook at the root of your repo is still the single highest-leverage configuration point, because it lands in the system prompt every turn. Conventions go here: package manager, test framework, formatting, “never touch /legacy,” “always use our logger.” Two hard-won lessons:

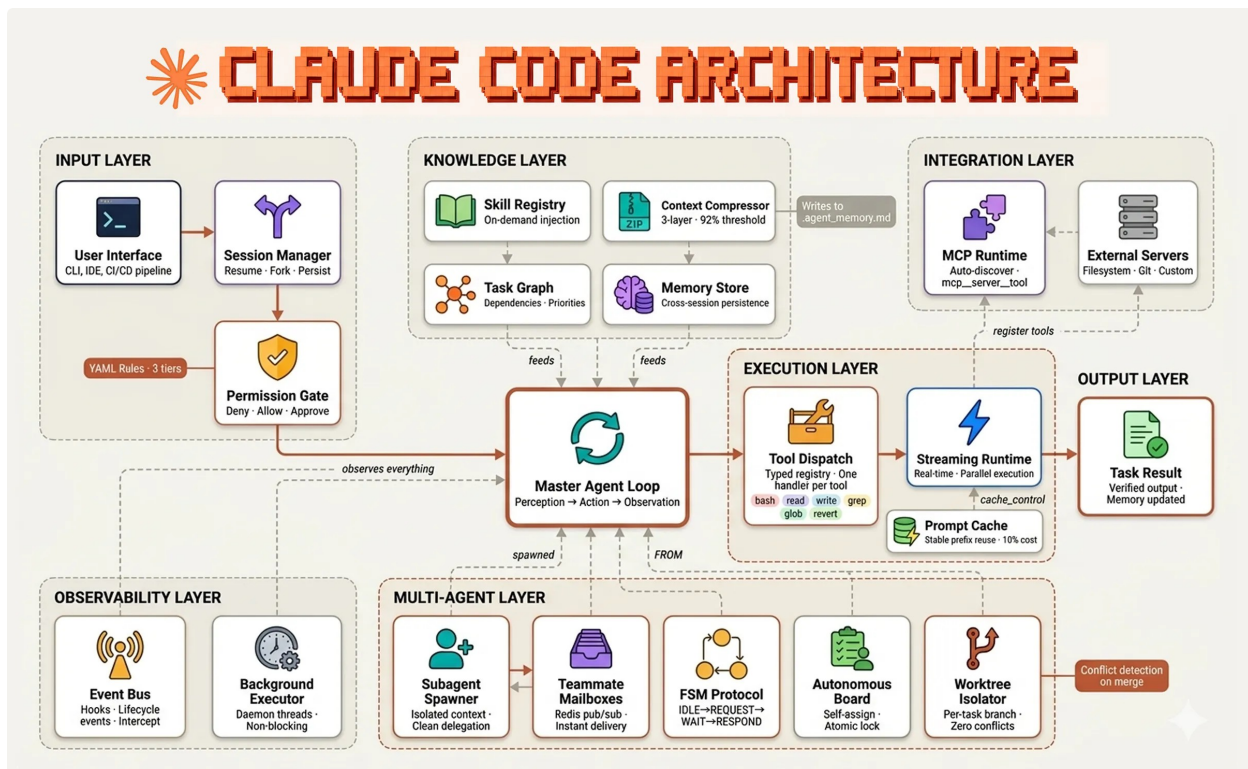
Keep it short. HumanLayer keeps theirs under 60 lines. Every line is competing for attention, and more rules make each rule matter less. Pilot’s checklist, not style guide.

Earn each line. Rules should trace to a specific past failure or a hard external constraint. If they don’t, they’re noise. Ratchet; don’t brainstorm.

Same discipline applies to tools. Each tool’s name, description, and schema gets stamped into the prompt every request. Ten focused tools outperform fifty overlapping ones because the model can hold the menu in its head. HumanLayer also flags a real security concern here: tool descriptions populate the prompt, so any MCP server you install is trusted text the model will read. A sloppy or malicious MCP can prompt-inject your agent before you’ve typed anything.

What this looks like in production

The clearest public picture I’ve seen of a mature harness is Fareed Khan’s (estimated) breakdown of [Claude Code’s architecture](#), and it’s worth sitting with the diagram for a minute.



Almost every concept from the previous section shows up on this diagram as a named component. Context injection is the knowledge layer. Loop state lives in the memory store and the worktree isolator. Destructive-action hooks sit behind the permission gate. Subagent context firewalls are the entire multi-agent layer. The tool dispatch registry is where MCP servers and bash both plug in. Khan's argument is the same as Viv's, just worked through a shipping product: Claude Code's trajectory is about the harness at least as much as about the model underneath it.

Harnesses don't shrink, they move

One of the better observations in the Anthropic write-up is that as models improve, the space of interesting harness combinations doesn't shrink. It moves.

The naive story is that better models make harnesses obsolete. If the model can plan, no planner. If the model is coherent at long horizons, no context resets. And yes, Opus 4.6 largely killed the context-anxiety failure mode (Sonnet 4.5 used to wrap up work prematurely as it approached what it thought was its context limit), which means a whole class of anxiety-mitigation scaffolding I was writing six months ago is now dead

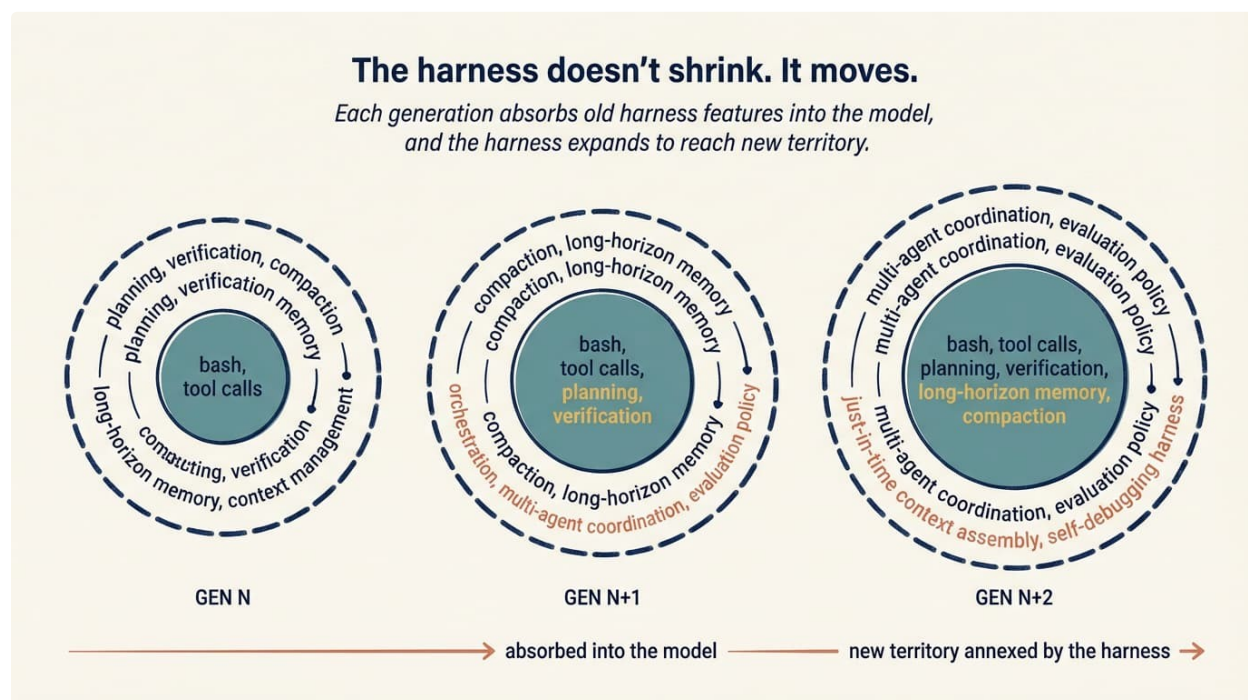
code.

But the ceiling moved with the model. Tasks that were unreachable are in play, and they have their own failure modes. The anxiety scaffolding goes away, and in its place you need a multi-day memory policy, or a harness that coordinates three specialized agents, or evaluators for design quality in generated UIs. The assumptions shift, and so does the scaffolding that encodes them.

Anthropic puts it cleanly: *“every component in a harness encodes an assumption about what the model can’t do on its own.”* When the model gets better at something, that component becomes load-bearing for nothing and should come out. When the model unlocks something new, new scaffolding is needed to reach the new ceiling.

The model-harness training loop

The other thing that’s happening, which Viv names explicitly, is a feedback loop between harness design and model training.



Today’s agent products are post-trained with harnesses in the loop. The model gets specifically better at the actions the harness designers think it should be good at: filesystem operations, bash, planning, subagent dispatch. That’s why Opus 4.6 feels different inside Claude Code than inside someone else’s harness, and it’s why changing a tool’s logic sometimes causes strange regressions. A genuinely general model

wouldn't care whether you used `apply_patch` or `str_replace`, but co-training creates overfitting.

The practical implication is twofold. A harness is a living system, not a config file you set up once. And the "best" harness isn't necessarily the one the model was trained inside; it's the one designed for your task. Viv's Top 30 to Top 5 Terminal Bench jump is the clearest proof point I've seen.

Harness-as-a-Service

Viv's other contribution is the [HaaS](#) framing: Harness-as-a-Service. The observation is that we're moving from building on LLM APIs (which give you a completion) to building on harness APIs (which give you a runtime). The Claude Agent SDK, the Codex SDK, and the OpenAI Agents SDK all point in the same direction. You get the loop, the tools, the context management, the hooks, and the sandbox primitives out of the box, and you customize them.

The shift matters because the default path used to be: build your own loop, wire up your own tool-calling, handle your own conversation state, invent your own approval flow. Now the default path is: pick a harness framework, configure it along the four pillars (system prompt, tools, context, subagents), and put the rest of your effort into domain-specific prompt and tool design.

That's what makes "skill issue" tractable. You're not rebuilding an agent from scratch every time something goes wrong. You're tuning a configuration surface that's already well-factored.

Viv's line on this is also the best argument for starting messy: *"good agent building is an exercise in iteration. You can't do iterations if you don't have a v0.1."*

Where this is going

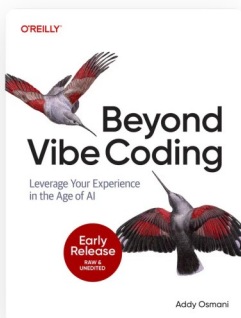
Look at the top coding agents side by side (Claude Code, Cursor, Codex, Aider, Cline)

and they look more like each other than their underlying models do. The models are different. The harness patterns are converging. I don't think that's an accident. It's the industry slowly finding the load-bearing pieces of scaffolding that turn a generative model into something that can ship.

Viv's framing of the open problems is the one I find most exciting: orchestrating many agents working in parallel on a shared codebase; agents that analyze their own traces to identify and fix harness-level failure modes; harnesses that dynamically assemble the right tools and context just-in-time for a given task instead of being pre-configured at startup.

That last one, in particular, feels like where harnesses stop being static config and start becoming something closer to a compiler.

ENJOYED THIS?



Go deeper in *Beyond Vibe Coding*

My O'Reilly book on AI-assisted and agentic engineering: specs, harnesses, evals, context, and shipping production-grade software with AI.

Read the book



Addy Osmani is an engineering and evangelism leader who spent over 14 years at Google leading developer experience across Chrome and, in recent years, AI (Gemini, coding agents, and agentic engineering), most recently as a Director at Google Cloud AI.



Tweet



Bluesky



Mastodon



Threads



LinkedIn



Share

Want more? **Subscribe to my free newsletter:**

example@gmail.com

Subscribe

Disclaimer: The views and opinions expressed on this site are my own.

© Copyright 2026 Addy Osmani